

Complexity Analysis

1. Overview

Complexity analysis is the study of how an algorithm's resource usage grows as the input size grows.

In interviews, the input size is usually written as n , and we mainly discuss:

- Time complexity: how the running time grows.
- Space complexity: how much extra memory the algorithm uses.
- Big-O, Big-Theta, Big-Omega: mathematical ways to describe growth.

Definition

Complexity analysis answers questions like:

If the input becomes 10 times larger, how much slower or more memory-heavy does the algorithm become?

It does not usually care about exact machine time like 2.3 ms. Instead, it focuses on growth patterns:

Complexity	Meaning
$O(1)$	Constant growth
$O(\log n)$	Logarithmic growth
$O(n)$	Linear growth
$O(n \log n)$	Linearithmic growth
$O(n^2)$	Quadratic growth
$O(2^n)$	Exponential growth
$O(n!)$	Factorial growth

Why It Matters

Two solutions can both work for small inputs but behave very differently for large inputs.

Example:

```
// O(n)
for (int i = 0; i < n; i++) {
    cout << arr[i];
}

// O(n^2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << arr[i] << arr[j];
    }
}
```

```
}  
}
```

For $n = 100000$, the first may be fine, while the second is usually impossible.

Where It Is Used in Real Systems

Complexity analysis is used in:

- Databases: choosing indexes and query plans.
- Operating systems: scheduling, memory allocation, page replacement.
- Backend servers: request handling, caching, rate limiting.
- Search engines: ranking, indexing, graph traversal.
- Browsers: DOM updates, JavaScript execution, rendering.
- Distributed systems: replication, consensus, load balancing.
- Application code: choosing between arrays, hash maps, heaps, trees, tries, and graphs.

Why Interviewers Ask About It

Interviewers ask complexity analysis because it shows whether you can:

- Predict if your code will pass constraints.
- Choose the right data structure.
- Compare multiple approaches.
- Reason about loops, recursion, and memory.
- Optimize without guessing.
- Explain trade-offs clearly.

In online assessments, complexity often decides whether a solution gets accepted or gets TLE.

2. Core Idea

The core idea is:

Count how the number of important operations grows with input size.

You do not count every CPU instruction. You count dominant operations.

Intuition

Suppose you are searching for a name in a phone book.

If you check every name one by one, the work grows linearly:

```
n names -> up to n checks -> O(n)
```

If the phone book is sorted and you repeatedly open the middle page, the search space halves each time:

```
n -> n/2 -> n/4 -> n/8 -> ... -> 1
```

That is logarithmic:

```
O(log n)
```

Real-World Analogy

Imagine finding a word in a dictionary.

Method	Behavior	Complexity
Read every word	Slow for large dictionaries	$O(n)$
Open middle, decide left/right	Very fast	$O(\log n)$
Ask someone who has a direct index	Immediate lookup	Average $O(1)$

Small Example

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

Step-by-step:

1. Loop starts at $i = 0$.
2. Loop runs while $i < n$.
3. $i++$ increases by 1.
4. The body runs exactly n times.
5. Each body operation is constant time.

So time complexity is:

```
O(n)
```

Extra memory:

```
sum, i -> constant variables -> O(1)
```

Space complexity is:

$O(1)$

Dominant Term Rule

When analyzing complexity, keep the fastest-growing term and drop constants.

 $3n^2 + 10n + 50 \rightarrow O(n^2)$

Why?

For large n , n^2 dominates n and constants.

3. Important Subtopics

3.1 Big-O

What It Means

Big-O describes an upper bound.

It answers:

The algorithm will not grow worse than this rate, up to constant factors.

Example:

Binary search = $O(\log n)$

This means binary search grows at most logarithmically.

Why It Matters

Big-O is the most common notation in interviews because interviewers usually care about worst-case scalability.

Example

```
for (int i = 0; i < n; i++) {  
    cout << i;  
}
```

The loop runs n times.

Time: $O(n)$

Common Interview Angle

Interviewers often ask:

What is the Big-O time complexity of your solution?

Expected answer:

The time complexity is $O(n)$ because I scan the array once, and the space complexity is $O(1)$ because I only use a few variables.

3.2 Big-Theta

What It Means

Big-Theta describes a tight bound.

It answers:

The algorithm grows exactly at this rate, up to constant factors.

If an algorithm is both $O(n)$ and $\Omega(n)$, then it is $\Theta(n)$.

Why It Matters

Big-Theta is more precise than Big-O.

Example

```
for (int i = 0; i < n; i++) {  
    cout << i;  
}
```

This always runs n times.

Time: $\Theta(n)$

Common Interview Angle

Interviewer may ask:

Is Big-O always tight?

Answer:

No. Big-O is only an upper bound. Big-Theta is a tight bound.

3.3 Big-Omega

What It Means

Big-Omega describes a lower bound.

It answers:

The algorithm takes at least this much work.

Why It Matters

It is useful for best-case analysis and theoretical limits.

Example

Linear search:

```
for (int i = 0; i < n; i++) {  
    if (arr[i] == target) return i;  
}
```

Best case:

Target is first element -> $\Omega(1)$

Worst case:

Target is last or absent -> $O(n)$

Common Interview Angle

Interviewer may ask:

What is the best-case complexity of linear search?

Answer:

$\Omega(1)$

3.4 Time Complexity Analysis

What It Means

Time complexity measures how the number of operations grows with input size.

Why It Matters

It helps predict whether a solution will pass input constraints.

Example

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        cout << i << j;  
    }  
}
```

Outer loop runs n times.

Inner loop runs n times for each outer loop.

Total:

$$n * n = n^2$$

Time:

$$O(n^2)$$

Common Interview Angle

They may give code and ask you to analyze it quickly.

3.5 Space Complexity Analysis

What It Means

Space complexity measures how extra memory grows with input size.

Usually, interviewers mean auxiliary space, not the input array itself, unless stated otherwise.

Why It Matters

Memory limits matter in online assessments and real systems.

Example

```
vector<int> copy;
for (int x : arr) {
    copy.push_back(x);
}
```

Extra vector stores n elements.

Space: $O(n)$

Common Interview Angle

They may ask:

Can you solve this in $O(1)$ extra space?

This usually means modifying input in-place or using only a few variables.

3.6 Worst, Average, and Best Case

What It Means

Different inputs of the same size can cause different running times.

Case	Meaning
Best case	Minimum time for any input of size n
Average case	Expected time over typical/random inputs
Worst case	Maximum time for any input of size n

Why It Matters

Worst case is important for guarantees. Average case is useful for expected practical performance.

Example

Linear search:

```
for (int i = 0; i < n; i++) {
    if (arr[i] == target) return i;
}
```

Case	Example	Time
Best	Target at index 0	$O(1)$

Case	Example	Time
Average	Target near middle	$O(n)$
Worst	Target absent or last	$O(n)$

Common Interview Angle

They may ask:

Why is hash map lookup $O(1)$ but sometimes $O(n)$?

Answer:

Average case is $O(1)$, but worst case can be $O(n)$ if many keys collide.

3.7 Amortized Analysis

What It Means

Amortized analysis studies average cost over a sequence of operations, even if some individual operations are expensive.

It is not the same as average-case probability. It gives a guarantee over a sequence.

Why It Matters

Many data structures have occasional expensive operations but are still efficient overall.

Example: Dynamic Array Push

When a dynamic array is full, it resizes:

Capacity: 1 -> 2 -> 4 -> 8 -> 16 -> ...

Most `push_back` operations are $O(1)$.

Occasionally, resizing copies all elements, costing $O(n)$.

But across many pushes, total copying is bounded:

$$1 + 2 + 4 + 8 + \dots + n < 2n$$

So each push is:

Amortized $O(1)$

Common Interview Angle

They may ask:

Why is vector `push_back` amortized $O(1)$?

Expected answer:

Because resizing happens rarely, and the total cost of all resizes over n insertions is $O(n)$, so average cost per insertion is $O(1)$.

3.8 Potential Method for Amortized Analysis

What It Means

The potential method assigns stored "credit" or "energy" to a data structure.

Cheap operations save credit. Expensive operations use saved credit.

Amortized cost:

Actual cost + change in potential

Formula:

Amortized cost = actual cost + $\Phi(\text{after}) - \Phi(\text{before})$

Φ is the potential function.

Why It Matters

It gives a formal way to prove amortized bounds.

Example: Dynamic Array

When the array has spare capacity, pushes are cheap but increase stored potential.

When resizing happens, the stored potential pays for copying old elements.

Simplified intuition:

Push without resize:
Actual cost = 1
Save some credit

Push with resize:

```
Actual cost = n
Use saved credit to pay copying cost
```

Common Interview Angle

Advanced interviewers may ask:

What is the difference between aggregate analysis and potential method?

Answer:

Aggregate analysis directly bounds total cost of n operations. Potential method assigns a potential function to the data structure and proves each operation has bounded amortized cost.

3.9 Recursion Complexity

What It Means

Recursion complexity measures time and space used by recursive calls.

For recursion, analyze:

- Number of calls.
- Work per call.
- Recursion depth.
- Extra memory from call stack.

Why It Matters

Many interview problems use DFS, backtracking, divide and conquer, and recursion.

Example

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

There are n recursive calls.

Each call does constant work.

```
Time: O(n)
Space: O(n) due to call stack
```

Common Interview Angle

They may ask:

Why is recursive factorial $O(n)$ space but iterative factorial $O(1)$ space?

Answer:

Recursive factorial stores n stack frames, while iterative factorial uses only a few variables.

3.10 Master Theorem Basics

What It Means

Master theorem solves recurrences of this form:

$$T(n) = aT(n / b) + f(n)$$

Where:

- a = number of subproblems.
- n / b = size of each subproblem.
- $f(n)$ = work done outside recursive calls.

Why It Matters

It quickly analyzes divide-and-conquer algorithms.

Example

Merge sort:

$$T(n) = 2T(n/2) + O(n)$$

Here:

$$\begin{aligned} a &= 2 \\ b &= 2 \\ f(n) &= n \\ n^{(\log_b a)} &= n^{(\log_2 2)} = n \end{aligned}$$

Since $f(n)$ matches $n^{(\log_b a)}$:

$$T(n) = O(n \log n)$$

Common Interview Angle

They may ask:

Why is merge sort $O(n \log n)$?

Expected answer:

There are $\log n$ levels of splitting, and each level does $O(n)$ total merge work, so total time is $O(n \log n)$.

3.11 Recurrence Solving

What It Means

A recurrence expresses recursive running time in terms of smaller inputs.

Why It Matters

Recursive algorithms are best analyzed through recurrences.

Example 1

$$T(n) = T(n - 1) + O(1)$$

Expanding:

$$\begin{aligned} T(n) &= T(n - 1) + 1 \\ &= T(n - 2) + 2 \\ &= T(n - 3) + 3 \\ &\dots \\ &= T(1) + n - 1 \end{aligned}$$

So:

$$T(n) = O(n)$$

Example 2

$$T(n) = 2T(n - 1) + O(1)$$

This doubles at every level.

$$T(n) = O(2^n)$$

Common Interview Angle

They may ask:

What recurrence represents binary search?

Answer:

$$T(n) = T(n/2) + O(1) = O(\log n)$$

3.12 Logarithmic Complexity Intuition

What It Means

Logarithmic complexity appears when the problem size shrinks by a constant factor each step.

Most commonly:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

Why It Matters

$O(\log n)$ algorithms are extremely efficient for large inputs.

Example

```
for (int i = 1; i <= n; i *= 2) {  
    cout << i;  
}
```

Values of i :

$$1, 2, 4, 8, 16, \dots, n$$

After k iterations:

$$i = 2^k$$

Stop when:

$$2^k > n$$

So:

$$k = \log_2(n)$$

Time:

$$O(\log n)$$

Common Interview Angle

They may ask:

Why is multiplying i by 2 logarithmic?

Answer:

Because the loop reaches n by powers of 2, so the number of iterations is the number of times we can double 1 before crossing n , which is $\log_2 n$.

3.13 Nested Loop With Dependent Bounds

What It Means

When an inner loop depends on the outer loop variable, do not blindly multiply $n * n$. Count total iterations carefully.

Example

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++) {}
```

For each i , the inner loop runs:

```
i = 0 -> n times  
i = 1 -> n - 1 times  
i = 2 -> n - 2 times  
...  
i = n - 1 -> 1 time
```

Total:

$$\begin{aligned} & n + (n - 1) + (n - 2) + \dots + 1 \\ &= n(n + 1) / 2 \\ &= O(n^2) \end{aligned}$$

Why It Matters

Many students incorrectly say the inner loop is always n , but the correct reasoning uses summation.

Common Interview Angle

Expected answer:

The exact number of iterations is $n(n + 1)/2$, so the time complexity is $\Theta(n^2)$.

4. Real-World Example

Database Query Search

Suppose a table has n users and you search by email.

Without Index

The database may scan every row:

```
Check row 1
Check row 2
Check row 3
...
Check row n
```

Time:

$O(n)$

With B-Tree Index

A database index works like a sorted tree.

Search repeatedly narrows the possible location.

Time is approximately:

$O(\log n)$

With Hash Index

For equality lookup, a hash index may provide average:

 $O(1)$

But worst case can degrade due to collisions.

Why This Matters

For 100 rows, all approaches may seem fast.

For 100 million rows:

Approach	Complexity	Practical Effect
Full table scan	$O(n)$	Slow
B-tree index	$O(\log n)$	Fast range and equality lookup
Hash index	Average $O(1)$	Fast equality lookup

This is exactly why backend engineers must understand complexity.

5. Diagrams / Mental Models

Growth Rate Comparison

Faster Slower
 $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(2^n) \rightarrow O(n!)$

Loop Analysis Mental Model

Single loop:
Count iterations.

Nested independent loops:
Multiply iterations.

Nested dependent loops:
Sum iterations.

Loop dividing or multiplying by constant:
Usually logarithmic.

Recursion:
Write recurrence.

Recursion Tree for Merge Sort

```

Level 0:           n                -> total work n
                  /  \
Level 1:       n/2    n/2          -> total work n
              /  \    /  \
Level 2:     n/4  n/4 n/4  n/4     -> total work n

Number of levels = log n
Work per level   = n

Total = n log n

```

Binary Search Shrinking Model

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

How many halvings?
 $\log_2(n)$

Dynamic Array Amortization Model

Push sequence:

Capacity 1: copy 1
Capacity 2: copy 2
Capacity 4: copy 4
Capacity 8: copy 8

Total copying:
 $1 + 2 + 4 + 8 + \dots + n < 2n$

Average per push = $O(1)$

6. Common Interview Questions

1. What is Big-O notation?

Big-O describes the upper bound of an algorithm's growth rate as input size increases.

Expected key points:

- Describes scalability, not exact runtime.
- Drops constants and lower-order terms.
- Often used for worst-case analysis.

Common mistakes:

- Saying Big-O is always exact.
- Including constants like $O(2n)$.
- Confusing runtime in seconds with growth rate.

2. Difference between Big-O, Big-Theta, and Big-Omega?

Notation	Meaning
Big-O	Upper bound
Big-Theta	Tight bound
Big-Omega	Lower bound

Expected key points:

- **O** means no worse than.
- **Theta** means exact asymptotic growth.
- **Omega** means at least.

Common mistakes:

- Treating all three as the same.
- Saying Big-O always means worst case.

3. What is the time complexity of this loop?

```
for (int i = 1; i <= n; i *= 2) {}
```

Answer:

$O(\log n)$

Reason:

i takes values 1, 2, 4, 8, After k iterations, $i = 2^k$. The loop stops when $2^k > n$, so $k = \log_2 n$.

Expected key points:

- Multiplication by constant causes logarithmic iterations.
- Base of log does not matter in Big-O.

Common mistakes:

- Saying $O(n)$ because there is one loop.
- Forgetting that i doubles.

4. What is the time complexity of this nested loop?

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++) {}
```

Answer:

$\Theta(n^2)$

Reason:

Total iterations:

$$n + (n - 1) + \dots + 1 = n(n + 1)/2$$

Expected key points:

- Inner loop depends on i .
- Use summation.
- Drop constant factor $1/2$.

Common mistakes:

- Saying $O(n)$ because inner loop decreases.
- Saying exact answer must include $/2$ in Big-O.

5. What is the difference between time and space complexity?

Time complexity measures growth of execution steps.

Space complexity measures growth of extra memory usage.

Expected key points:

- Recursion uses call stack.
- Input memory is usually excluded when discussing auxiliary space.
- Extra arrays, maps, sets, queues, recursion stack all count.

Common mistakes:

- Forgetting recursion stack.
- Counting only variables and ignoring data structures.

6. Why is binary search $O(\log n)$?

Binary search halves the search space every step.

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$$

Number of halvings is $\log_2 n$.

Expected key points:

- Requires sorted data.
- Each step discards half.
- Time is $O(\log n)$, space is $O(1)$ iterative or $O(\log n)$ recursive.

Common mistakes:

- Forgetting sorted array requirement.
- Saying recursive binary search is $O(1)$ space.

7. Why is merge sort $O(n \log n)$?

Merge sort splits the array into halves and merges at each level.

$$T(n) = 2T(n/2) + O(n)$$

There are $\log n$ levels, and each level does $O(n)$ merge work.

Total:

$$O(n \log n)$$

Expected key points:

- Divide into two halves.
- Merge work per level is linear.
- Recursion depth is $\log n$.

Common mistakes:

- Saying $O(\log n)$ because the array is divided.
- Ignoring merge cost.

8. What is amortized analysis?

Amortized analysis gives the average cost per operation over a sequence of operations, with a worst-case guarantee over the sequence.

Example:

```
vector push_back = amortized  $O(1)$ 
```

Expected key points:

- Some operations may be expensive.
- Total cost over many operations is bounded.
- Different from probabilistic average case.

Common mistakes:

- Saying amortized means average input.
- Saying every push is always worst-case $O(1)$.

9. What is the space complexity of DFS?

For graph DFS:

```
Time:  $O(V + E)$   
Space:  $O(V)$ 
```

Space comes from:

- Visited array or set.
- Recursion stack or explicit stack.

Expected key points:

- Graph traversal visits vertices and edges.
- Stack can grow to V .

Common mistakes:

- Saying $O(1)$ because no extra array is visible.
- Forgetting recursion stack.

10. What is the Master Theorem?

Master theorem solves recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

It compares $f(n)$ with:

$$n^{\log_b a}$$

Expected key points:

- Used for divide-and-conquer recurrences.
- Not every recurrence fits Master theorem.
- Common examples: binary search, merge sort.

Common mistakes:

- Applying it to $T(n) = T(n - 1) + n$.
- Forgetting the recurrence must be in the correct form.

11. What is the complexity of linear search?

Best case:

$O(1)$

Average case:

$O(n)$

Worst case:

$O(n)$

Expected key points:

- Best case when target is first.
- Worst case when target is absent or last.
- Average still linear.

Common mistakes:

- Saying average is $O(n/2)$ instead of $O(n)$.

12. What is the complexity of checking all pairs in an array?

Code:

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        check(arr[i], arr[j]);  
    }  
}
```

Answer:

$$O(n^2)$$

Exact number of pairs:

$$n(n - 1)/2$$

Expected key points:

- It checks every unordered pair once.
- Constant factor $1/2$ is ignored.

Common mistakes:

- Saying $O(n)$ because each pair is checked once.

7. Deep-Dive Questions

1. Is $O(n)$ always better than $O(n \log n)$?

Asymptotically, yes, for sufficiently large n , $O(n)$ grows slower than $O(n \log n)$.

But in practice, constants and hardware behavior matter for small inputs.

Example:

An $O(n \log n)$ algorithm with simple operations may beat an $O(n)$ algorithm with expensive hashing for small n .

Still, for interviews, compare asymptotic growth first.

2. Why is the base of logarithm ignored in Big-O?

Because logarithms of different bases differ by a constant factor.

$$\log_2 n = \log_{10} n / \log_{10} 2$$

$1 / \log_{10} 2$ is a constant.

So:

$$O(\log_2 n) = O(\log_{10} n) = O(\log n)$$

3. Why is quicksort average $O(n \log n)$ but worst $O(n^2)$?

Quicksort partitions around a pivot.

If partitions are balanced:

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

If pivot is always smallest or largest:

$$T(n) = T(n - 1) + O(n) = O(n^2)$$

Expected key points:

- Pivot quality determines recursion depth.
- Randomized pivot reduces chance of worst case.

4. How do you analyze backtracking complexity?

Backtracking usually explores a decision tree.

Steps:

1. Count choices at each level.
2. Count depth.
3. Multiply or express as branching factor.

Example: generating all subsets.

Each element has two choices: take or not take.

2^n subsets
Time: $O(2^n)$
Space: $O(n)$ recursion depth, excluding output

If output is counted:

Output space: $O(n * 2^n)$

5. How can an algorithm be $O(1)$ but still slow?

Big-O ignores constants.

An $O(1)$ operation may involve:

- Network call.
- Disk I/O.

- Large fixed-size matrix operation.
- Lock contention.
- Cache miss.

So Big-O describes growth, not exact speed.

8. Comparison Tables

Big-O vs Big-Theta vs Big-Omega

Notation	Describes	Meaning	Example
$O(g(n))$	Upper bound	At most this growth	Linear search worst case $O(n)$
$\Theta(g(n))$	Tight bound	Exactly this growth asymptotically	Single loop $\Theta(n)$
$\Omega(g(n))$	Lower bound	At least this growth	Linear search best case $\Omega(1)$

Time Complexity vs Space Complexity

Aspect	Time Complexity	Space Complexity
Measures	Operations	Memory
Common sources	Loops, recursion, comparisons	Arrays, maps, sets, call stack
Interview question	"How fast is it?"	"Can we reduce memory?"
Example	Sorting takes $O(n \log n)$	Merge sort uses $O(n)$ extra space

Worst vs Average vs Best Case

Case	Meaning	Example: Linear Search
Best	Minimum work	Target first, $O(1)$
Average	Expected typical work	Target around middle, $O(n)$
Worst	Maximum work	Target absent, $O(n)$

Big-O Growth Comparison

Complexity	Common Example	Scales Well?
$O(1)$	Array index access	Excellent
$O(\log n)$	Binary search	Excellent
$O(n)$	Linear scan	Good
$O(n \log n)$	Merge sort	Good
$O(n^2)$	All pairs	Bad for large n
$O(2^n)$	Subsets/backtracking	Very bad

Complexity	Common Example	Scales Well?
$O(n!)$	Permutations brute force	Extremely bad

Common Data Structure Complexities

Operation	Array	Sorted Array	Hash Map	Balanced BST
Access by index	$O(1)$	$O(1)$	N/A	N/A
Search	$O(n)$	$O(\log n)$	Average $O(1)$	$O(\log n)$
Insert	$O(n)$	$O(n)$	Average $O(1)$	$O(\log n)$
Delete	$O(n)$	$O(n)$	Average $O(1)$	$O(\log n)$
Ordered traversal	$O(n)$	$O(n)$	Not ordered	$O(n)$

Recursion Patterns

Pattern	Recurrence	Complexity	Example
Decrease by 1	$T(n) = T(n-1) + O(1)$	$O(n)$	Factorial
Divide by 2	$T(n) = T(n/2) + O(1)$	$O(\log n)$	Binary search
Two halves plus merge	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	Merge sort
Two calls decreasing by 1	$T(n) = 2T(n-1) + O(1)$	$O(2^n)$	Naive recursion

9. Common Mistakes

- Saying every single loop is $O(n)$, even when $i *= 2$ makes it $O(\log n)$.
- Multiplying nested loops blindly without checking dependent bounds.
- Keeping constants in Big-O, such as $O(2n)$ or $O(n/2)$.
- Forgetting recursion stack in space complexity.
- Confusing average case with amortized analysis.
- Saying binary search is $O(\log n)$ without mentioning sorted input.
- Assuming hash map operations are always worst-case $O(1)$.
- Ignoring output size in problems that generate all subsets, permutations, or combinations.
- Applying Master theorem to recurrences that do not match its form.
- Saying $O(n \log n)$ because the algorithm "has recursion" without checking work per level.
- Confusing number of recursive calls with recursion depth.
- Ignoring input constraints in online assessment problems.

10. Edge Cases / Special Cases

10.1 Constant Factors Can Matter Practically

Big-O ignores constants, but real systems do not.

$O(n)$ with heavy disk I/O may be slower than $O(n \log n)$ in memory for small n .

10.2 Output Size Can Dominate

Generating all subsets has at least $O(2^n)$ time because there are 2^n outputs.

Generating all permutations has at least $O(n!)$ outputs.

No clever algorithm can print exponential output in linear time.

10.3 Recursive Space Is Not Always Same as Time

Merge sort:

Time: $O(n \log n)$
Space: $O(n)$

Binary search recursive:

Time: $O(\log n)$
Space: $O(\log n)$

Binary search iterative:

Time: $O(\log n)$
Space: $O(1)$

10.4 Hash Map Worst Case

Hash map operations are average $O(1)$, but worst case may be $O(n)$ if many collisions occur.

Some implementations reduce this risk using resizing, randomization, or tree buckets.

10.5 Sorting Lower Bound

Comparison-based sorting has lower bound:

$\Omega(n \log n)$

But non-comparison sorts like counting sort can be faster under special constraints:

$O(n + k)$

where k is the range of values.

10.6 Logarithmic Loops Can Increase or Decrease

Both are logarithmic:

```
for (int i = 1; i <= n; i *= 2) {}
```

```
for (int i = n; i > 0; i /= 2) {}
```

One grows toward n ; the other shrinks toward 1 .

10.7 Multiple Variables

If input has two sizes, use both.

Example:

```
for (int i = 0; i < n; i++) {}  
for (int j = 0; j < m; j++) {}
```

Time:

 $O(n + m)$

Nested:

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++) {}
```

Time:

 $O(nm)$

11. How to Explain in Interview

Complexity analysis is how I estimate how an algorithm's time and memory grow with input size. I focus on the dominant operation, ignore constants and lower-order terms, and express the result using Big-O for upper bounds, Big-Theta for tight bounds, and Big-Omega for lower bounds. For loops, I count iterations; for nested loops, I multiply or sum depending on the bounds; for recursion, I write a recurrence or use a recursion tree/Master theorem. I also mention space separately, including extra data structures and recursion stack.

12. Quick Revision Notes

Key Definitions

Term	Meaning
Time complexity	Growth of operations
Space complexity	Growth of extra memory
Big-O	Upper bound
Big-Theta	Tight bound
Big-Omega	Lower bound
Worst case	Maximum work
Average case	Expected work
Best case	Minimum work
Amortized	Average per operation over a sequence

Important Points

- Drop constants: $O(2n)$ becomes $O(n)$.
- Drop lower terms: $O(n^2 + n)$ becomes $O(n^2)$.
- One loop increasing by 1 is often $O(n)$.
- Loop multiplying/dividing by 2 is often $O(\log n)$.
- Two independent nested loops are often $O(n^2)$.
- Dependent nested loops often require summation.
- Recursive algorithms need stack-space analysis.
- Output size must be counted when output is large.

Common Comparisons

Concept	Common Confusion	Correct Idea
Big-O vs Theta	Both mean exact time	Big-O upper bound, Theta tight bound
Average vs Amortized	Same thing	Average is probabilistic; amortized is over sequence
Time vs Space	Only time matters	Memory can also fail constraints
Recursion calls vs depth	Same thing	Calls affect time; depth affects stack
Hash map lookup	Always $O(1)$	Average $O(1)$, worst can be $O(n)$

Must-Remember Facts

- Binary search: $O(\log n)$.
- Merge sort: $O(n \log n)$ time, $O(n)$ extra space.
- Quick sort: average $O(n \log n)$, worst $O(n^2)$.
- BFS/DFS: $O(V + E)$ time.
- Dynamic array push: amortized $O(1)$.
- All subsets: $O(2^n)$.
- All permutations: $O(n!)$.

Interview Traps

- `for (i = 1; i <= n; i *= 2)` is $O(\log n)$, not $O(n)$.
- `for (j = i; j < n; j++)` inside `i` loop is still $O(n^2)$, but because of summation.
- Recursive functions use stack space.
- Sorting is not always $O(n \log n)$ if constraints allow counting sort or radix sort.
- Big-O does not measure exact seconds.

13. Practice Tasks

Task 1: Analyze Simple Loops

Find time complexity:

```
for (int i = 0; i < n; i++) {}
```

Answer:

$O(n)$

```
for (int i = 1; i <= n; i *= 2) {}
```

Answer:

$O(\log n)$

```
for (int i = n; i > 0; i /= 2) {}
```

Answer:

$O(\log n)$

Task 2: Analyze Nested Loops

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {}  
}
```

Answer:

$O(n^2)$

```
for (int i = 0; i < n; i++) {  
    for (int j = i; j < n; j++) {}  
}
```

Answer:

$O(n^2)$

Reason:

$$n + (n - 1) + \dots + 1 = n(n + 1)/2$$

Task 3: Analyze Recursion

```
int f(int n) {  
    if (n <= 1) return 1;  
    return f(n - 1) + 1;  
}
```

Answer:

Time: $O(n)$
Space: $O(n)$

```
int f(int n) {  
    if (n <= 1) return 1;  
    return f(n / 2) + 1;  
}
```

Answer:

Time: $O(\log n)$
Space: $O(\log n)$

Task 4: Write Recurrences

Write recurrence and solve:

```
void mergeSort(vector<int>& arr, int l, int r) {  
    if (l >= r) return;  
    int mid = (l + r) / 2;  
    mergeSort(arr, l, mid);  
    mergeSort(arr, mid + 1, r);  
    merge(arr, l, mid, r);  
}
```

Answer:

$T(n) = 2T(n/2) + O(n)$
 $T(n) = O(n \log n)$

Task 5: Implement and Compare

Implement two approaches for checking whether an array has duplicates.

Approach 1:

```
bool hasDuplicateBrute(vector<int>& arr) {  
    int n = arr.size();  
    for (int i = 0; i < n; i++) {  
        for (int j = i + 1; j < n; j++) {  
            if (arr[i] == arr[j]) return true;  
        }  
    }  
    return false;  
}
```

Complexity:

```
Time:  $O(n^2)$   
Space:  $O(1)$ 
```

Approach 2:

```
bool hasDuplicateHash(vector<int>& arr) {  
    unordered_set<int> seen;  
    for (int x : arr) {  
        if (seen.count(x)) return true;  
        seen.insert(x);  
    }  
    return false;  
}
```

Complexity:

```
Average time:  $O(n)$   
Worst time:  $O(n^2)$  depending on hash collisions  
Space:  $O(n)$ 
```

Interview lesson:

Hashing improves time by using extra space.

Task 6: Amortized Analysis Exercise

Suppose a dynamic array doubles capacity when full.

Insert 8 elements into an initially empty array.

Track resize costs:

```
Insert 1: capacity grows to 1, copy 0  
Insert 2: capacity grows to 2, copy 1  
Insert 3: capacity grows to 4, copy 2  
Insert 5: capacity grows to 8, copy 4
```

Total copy cost:

$$0 + 1 + 2 + 4 = 7$$

Total insertion base cost:

8

Total:

$$15 = O(n)$$

Amortized per insertion:

$$O(1)$$

14. Final Cheat Sheet

Core Definition

Complexity analysis measures how an algorithm's time and memory usage grow as input size grows.

Why It Matters

It helps you choose efficient solutions, pass online assessment constraints, and explain trade-offs clearly in interviews.

Most Asked Questions

Question	Quick Answer
What is Big-O?	Upper bound on growth
Big-O vs Theta?	Big-O upper bound, Theta tight bound
Why binary search is $O(\log n)$?	Search space halves each step
Why merge sort is $O(n \log n)$?	$\log n$ levels, n work per level
What is amortized $O(1)$?	Average per operation over sequence is constant
What is recursion space?	Maximum call stack depth
What is hash map lookup?	Average $O(1)$, worst $O(n)$

Common Comparisons

Concept	Remember
Big-O	Upper bound
Big-Theta	Tight bound
Big-Omega	Lower bound
Worst case	Maximum work
Average case	Expected work
Best case	Minimum work
Amortized	Cost averaged over operation sequence

Fast Code Analysis Patterns

```
for (int i = 0; i < n; i++) {}
```

$O(n)$

```
for (int i = 1; i <= n; i *= 2) {}
```

$O(\log n)$

```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++) {}
```

$O(n^2)$

```
for (int i = 0; i < n; i++)
  for (int j = i; j < n; j++) {}
```

$n + (n - 1) + \dots + 1 = O(n^2)$

```
T(n) = T(n/2) + O(1)      -> O(log n)
T(n) = T(n-1) + O(1)      -> O(n)
T(n) = 2T(n/2) + O(n)     -> O(n log n)
T(n) = 2T(n-1) + O(1)     -> O(2^n)
```

One-Line Interview Answer

Complexity analysis is how I describe how an algorithm's time and extra memory grow with input size; I count dominant operations, ignore constants, use Big-O/Theta/Omega for bounds, and analyze loops, recursion, data structures, and amortized behavior separately.